

SentinelAPI

What we built, how it works, and why it matters

AMIHACKS 2026 — Track C (Deep-Tech)

September 24, 2026

“We built a tool that finds a website’s hidden security holes automatically — before a hacker does.”

1. The Problem

Apps talk to servers through “doors” called **APIs**. Every time you log in, check a balance, or open a profile, your app is knocking on one of these doors. When a door is built carelessly, two things go wrong:

1. **The open door.** The server hands over *someone else’s* data when you ask — because nobody checked who is asking.
2. **The oversharing door.** The server sends back *too much* — passwords, card numbers, private details the app never needed.

Hackers find these mistakes every day. Companies usually notice only *after* the damage is done, because checking by hand is slow and expensive.

2. What We Built

SentinelAPI is an automatic security inspector. You point it at an app’s doors, and it tells you exactly which ones are unsafe — in plain words, with a one-line proof, so anyone can act on it.

In one sentence

It creates two imaginary users, checks whether one can see the other’s secrets, and checks whether the server is sharing more than it promised.

3. How It Works

Five simple steps:

1. **We built a practice app** with known flaws, like a training dummy for the inspector to test itself on.

2. **The inspector creates two imaginary users** — let's call them Alice and Bob.
3. **It pretends to be Alice and asks for Bob's stuff.** If the server says “*sure, here's Bob's data*” — that's a red flag. We call it the *open-door* problem.
4. **It compares the promise with reality.** The server's own rulebook says what it will share; the inspector checks what it *actually* shares. Extra secrets — passwords, card numbers — are another red flag. We call it the *oversharing* problem.
5. **It writes a report.** Every problem gets a colour badge (**red** = serious, **orange** = important) plus a copy-paste command that proves it.

4. The Tech Stack

The building blocks, in plain terms:

Tool	What it does for us
Python + FastAPI	The practice app, and the “brain” of the inspector
httpx	How the inspector knocks on the app's doors
OpenAPI	The server's rulebook the inspector reads
HTML + CSS	The clean report page with the colour badges
Rust (planned)	A faster inspector engine for very large apps

5. What's New About It

We test, we don't guess

Most scanners throw random junk at a server and hope something breaks. We do something smarter: we create **two real users** and check whether one can see the other's data. That is a *real test*, not a guess.

We use the server's own rulebook

We compare what the server *promised* to share against what it actually shares. The difference between promise and reality is where the leak is.

Every finding comes with proof

No vague warnings. Each problem includes a copy-paste command anyone can run to see the leak with their own eyes — so there are *no false alarms*.

Runs anywhere, costs nothing

The whole thing works offline with free, open tools — no paid services, no cloud, no internet needed for the demo.

Built to grow

The engine is designed so we can swap the Python brain for a faster **Rust** one later, without changing anything else.

6. How We Show It

The live demo takes about **30 seconds**:

1. Start the practice app.
2. Run the inspector.
3. Open the report — a clean page with **red** and **orange** cards, each one saying *what* is wrong, *why*, and *how to prove it*.
4. Run the proof command on screen — and watch Bob's private data appear while logged in as Alice.

The result: a tool that finds security holes automatically, explains them in plain words, and proves them — before a hacker does.